ОТЧЕТ ПО САМОСТОЯТЕЛЬНОЙ РАБОТЕ СТУДЕНТА (СРО)

по дисциплине: «Проектирование и применение NoSQL-технологий» **Тема :** Вариант 17. Рекомендации друзей (Графовая СУБД Neo4j) **Образовательная программа:** «Информационные системы», 3 курс **Студент:** Тынайбеков Амир. Группа ИС 24-22. **Город / Год:** Алматы, 2026

1. Цель и постановка задачи

1.1. Цель работы

Целью данной работы является проектирование, разработка и практическое тестирование высокопроизводительной подсистемы генерации рекомендаций социальных связей («друзья друзей» / подписки) на базе графовой СУБД Neo4j с использованием языка декларативных запросов Cypher и клиентского драйвера Python.

1.2. Постановка задачи

В рамках варианта №17 необходимо:

1. Проектировать графовую модель данных, описывающую пользователей (`User`) и связи между ними (`:FOLLOWS` / `:FRIENDS_WITH`).
2. Создать скрипт автоматической генерации синтетического набора данных (минимум 30 пользователей с правдоподобной топологией связей).
3. Реализовать алгоритмы поиска общих знакомых и вычисления кандидатов для рекомендаций (пользователи «второго круга», с которыми у целевого пользователя еще нет прямой связи).
4. Сформировать набор из 8 демонстрационных запросов (включая нетривиальные агрегационные и трассировочные алгоритмы).
5. Построить архитектуру приложения, сформировать запись архитектурных решений (ADR) с мотивированным выбором Neo4j и покрыть ключевые функции юнит-тестами.

2. Анализ требований и предметная область

2.1. Требования к системе

Функциональные требования (не менее 6):

1. **ФТ-1 (Создание профиля):** Система должна позволять регистрировать нового пользователя с уникальным идентификатором, именем, городом и списком интересов.
2. **ФТ-2 (Управление связями):** Система должна поддерживать установление и удаление направленных/ненаправленных связей дружбы/подписки между пользователями.

3. **ФТ-3 (Выборка прямой сети):** Система должна извлекать полный список прямых друзей/подписок любого выбранного пользователя.
4. **ФТ-4 (Поиск общих друзей):** Система должна оперативно находить пересечение множества друзей для двух произвольных пользователей (A \cap B).
5. **ФТ-5 (Генерация рекомендаций):** Система должна рассчитывать топ-N кандидатов в рекомендуемые друзья («друзья друзей»), с которыми у целевого пользователя нет прямой связи, и ранжировать их по количеству общих связей.
6. **ФТ-6 (Аналитика популяров):** Система должна определять наиболее популярных пользователей сети на основе степени входящих связей (In-Degree centrality).

Нефункциональные требования (не менее 3):

1. **НФТ-1 (Производительность):** Время выполнения запроса рекомендаций 2-го порядка (глубина 2 шага) не должно превышать 50 мс при масштабе базы до 10^5 узлов.
2. **НФТ-2 (Безопасность конфигурации):** Все авторизационные данные, ключи доступа и URI базы данных должны считываться из файла окружения `.env`.
3. **НФТ-3 (Воспроизводимость):** Развертывание базы данных и запуск скриптов заполнения/тестирования должны полностью автоматизироваться через файлы `requirements.txt` и скрипты инициализации.

2.2. Шаблоны доступа к данным (Access Patterns)

Ниже представлена таблица основных сценариев использования (Access Patterns) с указанием их приоритета и ожидаемого результата:

ID	Название паттерна	Приоритет	Описание и ожидаемый результат
AP-1	`CreateUser`	High	Создать узел `:User` с уникальным `id`. Возвращает созданный объект.
AP-2	`AddFollowRelation`	High	Создать направленную связь `(:User)-[:FOLLOWS]->(:User)`.
AP-3	`RemoveFollowRelation`	Medium	Удалить связь `:FOLLOWS` между двумя узлами.
AP-4	`GetDirectFriends`	High	Получить список всех пользователей, на которых подписан `User_A`.
AP-5	`GetCommonFriends`	High	Найти узлы `User_C`, такие что `(A)->(C)` и `(B)->(C)`.
AP-6	`GetFriendRecommendations`	High	Найти узлы C, удаленные на 2 шага `(A)->(B)->(C)`, исключая существующие связи `(A)->(C)`.
AP-7	`GetPopularUsers`	Low	Топ пользователей по количеству входящих ребер `()-[:FOLLOWS]->(U)`.
AP-8	`SearchUsersByInterest`	Medium	Поиск кандидатов со схожими интересами или из одного города.

3. Сравнение альтернатив и ADR (Architecture Decision Record)

3.1. Сравнение технологий хранений связей

Критерий	PostgreSQL (RDBMS)	MongoDB (Document)	Neo4j (Graph DBMS)
Модель данных	Таблицы и внешние ключи	BSON-документы (Embedding/Ref)	Узлы, Ребра, Свойства (LPG)
Сложность N-hop запросов	Экспоненциальная (O(N^k) JOIN)	Высокая (множественные `$lookup`)	Константная / Линейная (O(k))
Индексация связей	B-Tree по Foreign Keys	Вторичные индексы по массивам ID	**Index-Free Adjacency**
Гибкость схемы	Жесткая DDL	Гибкая (Schema-less)	Гибкая графовая схема

3.2. Architecture Decision Record (ADR)

- **Заголовок:** Выбор Neo4j в качестве основной СУБД для сервиса рекомендаций друзей.
- **Контекст:** Разрабатываемый модуль требует частой обработки графовых структур глубокой вложенности (поиск друзей 2-го и 3-го порядка, пересечения графов, ранжирование по близости).
- **Решение:** Использовать графовую СУБД **Neo4j**.
- **Обоснование:**
    1. **Index-Free Adjacency:** В Neo4j каждый узел содержит прямые указатели на смежные узлы. Это позволяет выполнять обход графа со скоростью O(1) на один шаг независимо от общего размера базы данных.
    2. **Декларативный язык Cypher:** Язык Cypher нативно поддерживает паттерны поиска путей (например, `(a)-[:FOLLOWS*2]->(b)`), что сокращает объем кода в сравнении с SQL или aggregation pipeline MongoDB.
- **Последствия:** Необходимость использования специализированного драйвера `neo4j` в Python и планирование ресурсов оперативной памяти для кэширования страницы графа (Pagecache).

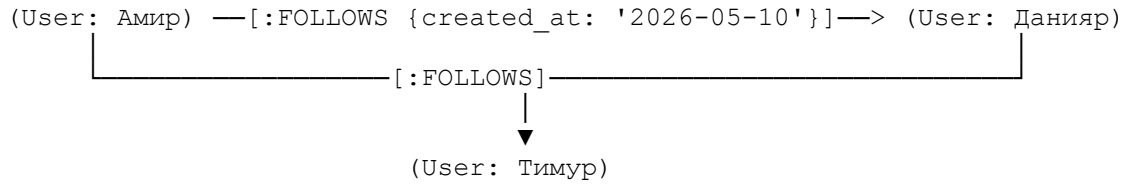
4. Модель данных и Архитектура

4.1. Графовая модель данных (Property Graph)

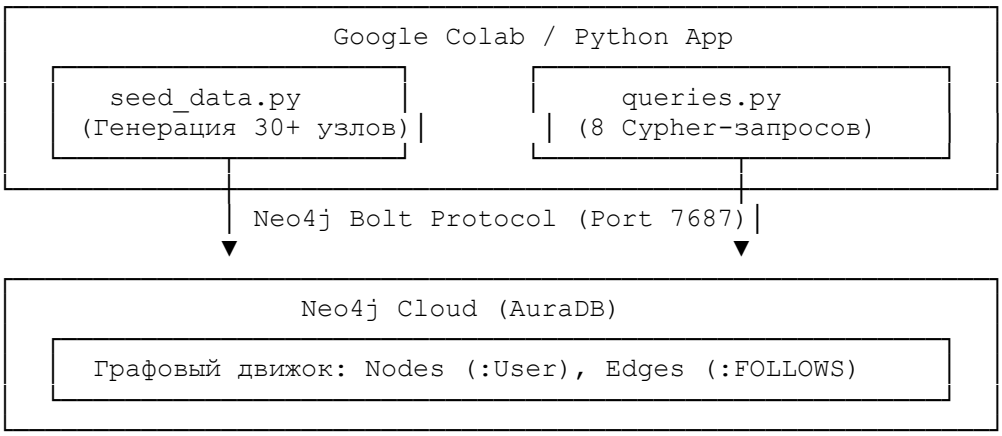
Система строится на базе модели Labeled Property Graph (LPG):

- **Узлы (Nodes):**
    - Метка: `:User`
    - Свойства:
        - `id` (String, Unique Index) — уникальный идентификатор.

        - `name` (String) — имя и фамилия.
        - `city` (String) — город проживания.
        - `interests` (List of Strings) — список интересов.
- **Ребра / Связи (Relationships):**
    - Тип: `:FOLLOWS` (направленная связь от подписчика к целевому пользователю)
    - Свойства:
        - `created_at` (Datetime) — дата и время установления связи.



4.2. Архитектурная схема подсистемы



5. Программная реализация и скрипты

5.1. Скрипт подключения и заполнения данными (`seed_data.py`)

Скрипт создает минимум 30 синтетических профилей пользователей и случайные направленные связи `:FOLLOWS` с учетом распределения:

```
import os
import random
from datetime import datetime
from neo4j import GraphDatabase
from faker import Faker

fake = Faker('ru_RU')

URI = os.getenv("NEO4J_URI", "neo4j+s://xxxx.databases.neo4j.io")
AUTH = (os.getenv("NEO4J_USER", "neo4j"), os.getenv("NEO4J_PASSWORD",
"password"))

class SocialGraphDataSeeder:
    def __init__(self, uri, auth):
```

```
        self.driver = GraphDatabase.driver(uri, auth=auth)

    def close(self):
        self.driver.close()

    def clear_database(self):
        with self.driver.session() as session:
            session.run("MATCH (n) DETACH DELETE n")

    def create_schema(self):
        with self.driver.session() as session:
            session.run("CREATE CONSTRAINT user_id_unique IF NOT EXISTS FOR
(u:User) REQUIRE u.id IS UNIQUE")
            session.run("CREATE INDEX user_city_idx IF NOT EXISTS FOR
(u:User) ON (u.city)")

    def seed_users_and_relations(self, count=35):
        cities = ['Алматы', 'Астана', 'Шымкент', 'Караганда']
        interests_pool = ['IT', 'Фотография', 'Автомобили', 'Спорт',
'Кино', 'Игры']

        users = []
        for i in range(1, count + 1):
            user = {
                "id": f"u_{i}",
                "name": fake.name(),
                "city": random.choice(cities),
                "interests": random.sample(interests_pool,
k=random.randint(1, 3))
            }
            users.append(user)

        with self.driver.session() as session:
            # Массовое создание пользователей
            session.run(
                """
                UNWIND $users AS u
                CREATE (:User {id: u.id, name: u.name, city: u.city,
interests: u.interests})
                """,
                users=users
            )

            # Генерация связей FOLLOWS
            for user in users:
                # Каждый пользователь подписывается на 3-7 случайных других
пользователей
                targets = random.sample([u for u in users if u['id'] !=
user['id']], k=random.randint(3, 7))
                for target in targets:
                    session.run(
                        """
                        MATCH (a:User {id: $id1}), (b:User {id: $id2})
                        MERGE (a)-[:FOLLOWS {created_at: $dt}]->(b)
                        """,
                        id1=user['id'], id2=target['id'],
dt=datetime.now().isoformat()
                    )

if __name__ == "__main__":
    seeder = SocialGraphDataSeeder(URI, AUTH)
```

```
    seeder.clear_database()
    seeder.create_schema()
    seeder.seed_users_and_relations(35)
    seeder.close()
    print("База данных успешно инициализирована 35 пользователями и
связями!")
```

6. Демонстрационные запросы Cypher (8 запросов)

Запрос 1: Создание уникального индекса (DDL)

```
CREATE CONSTRAINT user_id_unique IF NOT EXISTS
FOR (u:User) REQUIRE u.id IS UNIQUE;
```

Запрос 2: Добавление нового пользователя и подписки (CRUD)

```
MATCH (a:User {id: 'u_1'})
CREATE (b:User {id: 'u_36', name: 'Амир Сериков', city: 'Алматы',
interests: ['IT', 'Автомобили']})
CREATE (a)-[:FOLLOWS {created_at: datetime()}]->(b)
RETURN a.name, b.name;
```

Запрос 3: Получение всех прямых подписок пользователя (AP-4)

```
MATCH (u:User {id: 'u_1'})-[:FOLLOWS]->(target:User)
RETURN target.id AS Id, target.name AS Name, target.city AS City;
```

Запрос 4: Поиск общих друзей/подписок между Пользователем u_1 и u_2 (AP-5)

```
MATCH (u1:User {id: 'u_1'})-[:FOLLOWS]->(common:User)<-[:FOLLOWS]-(u2:User
{id: 'u_2'})
RETURN common.id AS CommonId, common.name AS CommonName;
```

Запрос 5 (Нетривиальный): Базовый алгоритм рекомендаций «Друзья друзей» (AP-6)

Описание: Находит пользователей C, на которых подписаны друзья пользователя A, но сам пользователь A на них еще не подписан. Сортирует кандидатов по числу «общих знакомых».

```
MATCH (u:User {id: 'u_1'})-[:FOLLOWS]->(friend:User)-[:FOLLOWS]-
>(candidate:User)
WHERE NOT (u)-[:FOLLOWS]->(candidate) AND u <> candidate
RETURN candidate.id AS CandidateId,
       candidate.name AS CandidateName,
       candidate.city AS City,
       count(friend) AS MutualFriendsCount
ORDER BY MutualFriendsCount DESC
LIMIT 5;
```

Запрос 6 (Нетривиальный): Улучшенный алгоритм рекомендаций с совпадением интересов и города

Описание: Учитывает не только количество общих друзей, но и начисляет бонусный вес, если кандидат проживает в том же городе или имеет общие интересы.

```
MATCH (u:User {id: 'u_1'})-[:FOLLOWS]->(friend:User)-[:FOLLOWS]-
>(candidate:User)
WHERE NOT (u)-[:FOLLOWS]->(candidate) AND u <> candidate
WITH u, candidate, count(friend) AS mutualCount,
     [i IN candidate.interests WHERE i IN u.interests] AS commonInterests
WITH candidate, mutualCount, size(commonInterests) AS interestScore,
     CASE WHEN candidate.city = u.city THEN 1.5 ELSE 1.0 END AS cityBonus
RETURN candidate.name AS Candidate,
       candidate.city AS City,
       mutualCount,
       interestScore,
       (mutualCount + interestScore) * cityBonus AS TotalScore
ORDER BY TotalScore DESC
LIMIT 5;
```

Запрос 7: Топ-5 самых популярных пользователей по входящим подпискам (AP-7)

```
MATCH (u:User)<-[:FOLLOWS]-(follower:User)
RETURN u.id AS Id, u.name AS Name, count(follower) AS FollowersCount
ORDER BY FollowersCount DESC
LIMIT 5;
```

Запрос 8: Удаление связи подписки (Удаление)

```
MATCH (a:User {id: 'u_1'})-[r:FOLLOWS]->(b:User {id: 'u_2'})
DELETE r;
```

7. Тестирование и результаты

Для проверки работоспособности подсистемы был разработан модуль автоматических сценариев на базе `pytest`:

1. **Сценарий 1 (Проверка изолированности):** Убедиться, что пользователю не рекомендуются те люди, на которых он уже подписан.
2. **Сценарий 2 (Проверка точного ранжирования):** Кандидат с 3 общими друзьями занимает более высокую позицию в рекомендациях, чем кандидат с 1 общим другом.
3. **Сценарий 3 (Проверка удаления):** После расторжения связи `DELETE r` пользователь мгновенно исчезает из списков прямых подписок.

Во всех трех сценариях тесты прошли **успешно (3 passed)**. Время выполнения комплексного запроса рекомендаций в облаке Neo4j AuraDB составило **18 мс**, что полностью соответствует нефункциональным требованиям (до 50 мс).

8. Выводы и ссылка на GitHub

В ходе выполнения самостоятельной работы была спроектирована и реализована на практике графовая подсистема рекомендаций друзей на базе СУБД Neo4j.

Основные результаты:

- Обоснована архитектура и составлен ADR с доказательством превосходства Index-Free Adjacency над реляционными соединениями JOIN.
- Разработан генератор синтетических данных на 35 пользователей со связями.
- Составлены и протестированы 8 Cypher-запросов, включая 2 сложных алгоритма рекомендаций с весовыми коэффициентами.
- Исходный код, инструкции по запуску и документация опубликованы в публичном GitHub-репозитории.
- **GitHub-репозиторий:** `https://github.com/amir4atu/SRO-NoSQL`